Reflective Journal – Chihuahua vs. Muffin CNN Classification

This lab gave me hands-on experience with convolutional neural networks (CNNs) for image classification. Unlike the previous workshop where I used a traditional neural network, this time I built a CNN to classify images like either Chihuahuas or muffins. During the activity, I learned how convolutional layers can automatically pick out important visual features from images, which helps the model perform better than a traditional neural network. Going through the notebook step by step also helped me understand the full deep learning process, from preparing the dataset to testing how well the model works.

The CNN architecture consisted of three convolutional layers with ReLU activation functions and max-pooling layers, which helped the model gradually learn more detailed features from the images while also reducing their size. After these layers, the features were flattened and passed through fully connected layers, along with a dropout layer, before making the final prediction. This setup is quite different from the traditional neural network used in the previous workshop because CNN keeps the spatial relationships between pixels and can automatically learn important visual features like edges, textures, and shapes. On the other hand, the traditional neural network required the image to be flattened into a one-dimensional vector, which caused it to lose a lot of important spatial information needed for accurate image recognition.

The model did well during training. I trained CNN for 15 epochs using the Adam optimizer with a learning rate of 0.001 and CrossEntropyLoss as the loss function. By the end, the model reached a training accuracy of 97.50% and a validation accuracy of 96.67%, which shows it was able to tell the difference between Chihuahuas and muffins accurately. In the validation set, only one image was classified incorrectly, while the rest were predicted correctly. This suggests that the model was able to generalize well, even though the dataset was fairly small. I also noticed that both the training and validation losses kept going down over time, which means the model was learning useful patterns instead of just guessing.

Compared with the traditional neural network from the previous workshop, I noticed that the CNN performed better overall. Even though the CNN took a bit longer to train because of the extra computations from the convolutional layers, it was much better at picking up important features from the images. The traditional neural network just used the pixel values after flattening the image, while the CNN was able to learn patterns like edges and shapes on its own.

Because of this, the CNN ended up with higher accuracy and worked better on the validation data. This helped me understand why CNNs are usually used for image classification tasks.

During the lab, I ran into a few challenges. At first, my Google Colab was using the CPU, which made everything run slower than expected. I fixed this by switching the runtime to use a T4 GPU, which helped speed things up a lot. I also saw a warning about the DataLoader worker processes, but after checking it, I realized it wasn't affecting how the notebook was running, so I left it as is. Another issue was filling in the missing values for the image size, which I set to 224 × 224. Lastly, I tried something extra by increasing the number of training epochs from 10 to 15 so I could see how it would affect the model's performance. This helped me understand how changing settings like epochs can impact the results instead of just sticking with the default values.

Image classification models like this CNN can be used in many real-world situations. For example, they can help doctors detect diseases from X-rays or MRI scans, help factories check products for defects, and help farmers identify plant diseases. They are also used in self-driving cars to recognize things like traffic signs and pedestrians. Other uses include security systems, tracking wildlife, and even stores. Overall, these examples show how computer vision can make tasks easier and help people make better decisions in different fields.

Even though CNNs work well, there are still some important ethical issues to think about when using image classification systems. One big concern is dataset bias. If the training data doesn't include enough variety, the model might not work well on images that are different from what it has seen before. Another issue is that the model can make mistakes, and in areas like healthcare or self-driving cars, those mistakes could be serious. Privacy is also important because image datasets might include personal information. It's important for developers to collect data responsibly, make sure they have permission to use it, and keep it secure. Overall, this shows that having a high accuracy isn't enough on its own—there also needs to be careful about testing, responsible data use, and human supervision.

Overall, this lab helped me understand how CNNs work and why they are effective for image classification. I gained hands-on experience with data preparation, model training, and evaluation, and saw that CNNs outperform traditional neural networks on image tasks.

References

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.

Szeliski, R. (2022). *Computer Vision: Algorithms and Applications* (2nd ed.). Springer.

PyTorch Contributors. (2025). *PyTorch Documentation*. https://pytorch.org/docs/stable/